

腾讯广告算法大赛面试准备文档

这份文档基于你提供的项目代码 (`main.py`, `model.py`, `infer.py`, `dataset.py` 等) 和简历描述整理, 旨在帮助你从代码实现细节和算法原理两方面深入准备面试。

1. 核心难点: 改进 BCE Loss 为 InfoNCE Loss

面试官可能问:

- Q: 为什么放弃 BCE Loss 而改用 InfoNCE Loss? 两者有什么区别?
- Q: 你的 InfoNCE 具体是怎么实现的? 正负样本怎么构造的?
- Q: 你提到了“批量负采样”, 具体是如何解决负样本稀疏问题的?

你的回答策略 (结合 `model.py`):

核心逻辑:

BCE (Binary Cross Entropy) 是 Point-wise 的, 独立判断每个样本是正还是负。而 InfoNCE 是 List-wise (或 Contrastive) 的, 它在一个 Batch 内通过对比正样本和多个负样本, 学习出更好的序列表示, 使其与正样本更近, 与负样本更远。

****代码级细节 (`model.py` -> `compute_infonce_loss`):**

1. 温度系数 (Temperature):

- 代码中定义了 `self.temp` (默认 0.07 或 0.02)。
- **作用:** 调节 Softmax 分布的平滑程度。温度越低, 分布越尖锐, 模型越关注难区分的负样本; 温度越高, 分布越平滑, 关注全局。
- **代码对应:** `logits = logits / self.temp`

2. 余弦相似度 (Cosine Similarity):

- 代码使用了 `F.cosine_similarity` 计算 `seq_embs` (用户序列表示) 和 `pos_embs` (正样本物品表示) 的相似度。
- **优势:** 相比点积, 余弦相似度归一化了向量长度, 让模型更关注向量方向 (语义一致性)。
- **代码对应:**

```
# 归一化确保是余弦相似度
seq_embs = self.normalize_embeddings(seq_embs)
pos_embs = self.normalize_embeddings(pos_embs)
pos_logits = F.cosine_similarity(seq_embs, pos_embs, dim=-1).unsqueeze(-1)
```

3. 批量负采样 (In-batch Negatives):

- **简历亮点:** "有效缓解 baseline 采样中负样本稀疏的问题"。
- **实现方式:** 不需要在 DataLoader 中显式采样大量负样本, 而是直接利用 **Batch 内其他用户的正样本** 作为当前用户的负样本。
- **代码对应:**

```
# 批内负采样: 当前 batch 的序列 embedding 与 负样本 embedding 矩阵相乘
neg_logits = torch.matmul(valid_seq_embs, valid_neg_embs.transpose(-1, -2))
# 排除对角线 (自己不能是自己的负样本)
mask = torch.eye(neg_logits.size(0)).bool()
neg_logits = neg_logits.masked_fill(mask, float('-inf'))
```

4. 难负样本挖掘 (Hard Negative Mining):

- 代码实现了 `topk` 策略, 只选取相似度最高的几个负样本 (即最容易混淆的“难”样本) 参与 Loss 计算, 提升判别能力。
- 代码对应: `hard_neg_logits, _ = torch.topk(neg_logits, self.neg_topk, dim=-1)`

2. 核心模型: 引入 HSTU 强化时序建模

面试官可能问:

- Q: HSTU (Hierarchical Sequential Transduction Unit) 相比于 SASRec (Transformer) 有什么改进?
- Q: 你是如何处理时间特征的? 为什么 SASRec 处理不好?

你的回答策略 (结合 `model.py` & `time_features.py`):

核心逻辑:

SASRec 使用标准的 Self-Attention, 计算复杂度是 $O(N^2)$, 且通常只加了绝对位置编码, 对时间间隔不敏感。HSTU 引入了门控机制 (Gating) 和聚合注意力, 更适合捕捉长序列中的动态兴趣变化。

代码级细节:

1. HSTU 的门控机制 (Gating):

- 在 `FlashMultiHeadAttention` 类中, 除了计算 Q, K, V, 还计算了一个 **U (Gate)** 矩阵。
- 最终输出是 Attention 结果与 Gate 的逐元素乘积。这是 HSTU 的标志性设计, 允许模型动态控制信息的流动, 过滤噪声。
- 代码对应 (`model.py`):

```
U = self.u_linear(value) # 额外的线性层生成 Gate
# ... Attention 计算 ...
attn_output = U * attn_output # 门控操作
```

2. 丰富的时序特征工程:

- 针对 Baseline 的不足, 你通过 `time_features.py` 构建了多维时间特征。
- 特征列表:
 - 200: 小时 (0-23) - 捕捉日内活跃规律
 - 201: 星期 (0-6) - 捕捉周度规律
 - 203: 对数时间间隔 ($\log(\text{gap})$) - 捕捉行为紧密度
 - 205: 时间衰减 ($\log(\text{delta}_t / \text{tau})$) - 赋予近期行为更高权重
 - 206: 与首行为的时间差 - 捕捉长期跨度

- **实现：** 这些特征在 `dataset.py` 的 `__getitem__` 中被计算，并在 `model.py` 中通过 `ITEM_TEMP_TIME_FEAT` 对应的 Embedding 层映射为向量，最后与物品 ID Embedding 拼接/相加。

3. 特征优化：SENet (Squeeze-and-Excitation)

面试官可能问：

- Q: SENet 原本是用于 CV 的，你怎么用到推荐系统里的？
- Q: 它的具体结构是怎样的？起了什么作用？

你的回答策略：

注意： 虽然 `model.py` 的当前版本主要展示了 HSTU 和 Embedding 变换，但面试中应基于简历描述回答实现逻辑。

实现逻辑：

SENet 用于特征维度的动态加权。在 Embedding 层之后，将所有 Field 的 Embedding 拼接。

1. **Squeeze (压缩):** 对特征进行 Global Average Pooling (或 Max Pooling)，将每个 Field 的 Embedding 压缩为一个标量，得到一个特征重要性向量 Z 。
2. **Excitation (激励):** 通过两个全连接层 (FC) + ReLU + Sigmoid，学习每个 Field 的权重 W 。
 - $Z \rightarrow FC_{reduce} \rightarrow ReLU \rightarrow FC_{expand} \rightarrow Sigmoid \rightarrow Weights$
3. **Reweight (加权):** 将权重 W 乘回原始 Embedding。

作用：

推荐系统中特征非常多（用户画像、物品属性、上下文），但不是所有特征在所有场景下都重要。SENet 让模型能够动态感知当前 Context 下哪些特征更关键（例如：周末时“星期”特征重要，深夜时“时间”特征重要）。

4. 工程优化：推理加速与数据加载

面试官可能问：

- Q: 这里的推理优化具体做了什么？为什么能提升效率？
- Q: 介绍一下你用的 FAISS。

你的回答策略 (结合 `infer.py` & `dataset.py`):

1. 数据处理前置 (`__getitem__`):

- **问题：** 原始训练中，数据预处理（如 Padding、时间特征计算）如果放在 Training Loop 主循环里，会阻塞 GPU 等待 CPU 数据。
- **优化：** 将所有复杂逻辑移至 `dataset.py` 的 `__getitem__`。结合 PyTorch `DataLoader` 的 `num_workers` (多进程)，利用 CPU 多核并行预处理数据。当 GPU 跑当前 Batch 时，CPU 已经在准备下一个 Batch 的数据了。
- **代码对应：** `dataset.py` 中详细的 `__getitem__` 逻辑（补全 `seq`, `pos`, `neg` 等）。

2. Batch-wise 推理 & 向量化:

- 推理时，不是对每个用户单独跑模型，而是组成 Batch（例如 `size=256`），一次性输入 GPU。

- 代码对应(infer.py): `test_loader` 的使用和 `model.predict` 的批次调用。

3. FAISS 向量检索:

- **原理:** 模型输出用户 Embedding 后, 需要从百万级物品库中找 Top-K。暴力计算 (逐一算余弦相似度) 太慢。
- **优化:** 使用 `faiss-gpu`。
 - 先离线通过 `model.save_item_emb` 计算所有 Item 的 Embedding 并保存 (`embedding.fbin`)。
 - 推理时, 使用 FAISS 构建索引 (如 HNSW 或 IVF), 将 $O(N)$ 的查找复杂度降至 $O(\log N)$ 或近似常数级。
- 代码对应(infer.py): `perform_python_faiss_search` 函数, 构建 `faiss.IndexHNSWFlat`, 设置 `metric_type=faiss.METRIC_INNER_PRODUCT` (内积/余弦)。

5. 项目成果总结 (Score: 0.0963, 提升 300%)

归因分析:

- **Baseline (0.023):** 简单的 SASRec + BCE Loss, 负样本采样不足, 时序特征单一。
- **提升来源:**
 1. **InfoNCE + 批内负采样:** 解决了“只在整个 Item 空间随机采负样本”导致的梯度更新效率低问题, 强迫模型区分难负样本。(贡献最大)
 2. **HSTU:** 更好的序列建模, 尤其在长序列和动态变化上优于 Transformer。
 3. **时序特征:** 显式的时间间隔和周期性特征, 帮模型捕获了“刚刚看过”和“周期性回看”的模式。

备用问题: 代码细节自查

- **Q: 你的代码里怎么处理变长序列的?**
 - A: 使用 `dataset.py` 中的 Padding 逻辑, 并在 `model.py` 中生成 `attention_mask` (`torch.tril`) 来屏蔽 Padding 位置和未来信息。
- **Q: 冷启动物品怎么处理?**
 - A: 代码中有 `_process_cold_start_feat`, 对于未见过的特征值使用默认值 (0) 或统计均值填充。
`infer.py` 中有详细的冷启动特征填充逻辑。